

Agentic Knowledge Graphs for Deterministic Code Generation

Engagement Structure

University Principal Investigator, Professor and Researcher(s)	Dr. Chris Brown (PI, Virginia Tech), Jason Cusati (GRA, Virginia Tech)
University	Virginia Tech, USA
eBay Co-Investigator(s)	Ramesh Periyathambi
eBay VP/Head of Department	
Grant Amount Requested	

Objectives

eBay’s codebase spans years of development across numerous teams, encoding recurring patterns—such as adding signals to ViewItem pages, implementing A/B test frameworks, or integrating with payment microservices—that developers re-implement manually each time. These tasks currently take 2–4 weeks per instance because developers must rediscover the relevant code locations, understand team-specific conventions, and ensure compatibility with the evolving dependency graph.

We propose to build an **agentic knowledge graph** (KG) that captures historical code patterns, team conventions, and project-level context from eBay’s codebase, enabling AI coding agents to generate solutions deterministically for recurring development tasks. Rather than treating each generation request as a stateless prompt, agents will query the KG for the most relevant prior patterns—which files changed, which APIs were used, what review feedback was given—and produce code that is consistent with past practice.

This is a strong candidate for academic research because the problem spans multiple open challenges: mining actionable patterns from large-scale industrial codebases, representing those patterns in a machine-readable KG, designing agentic workflows that use KG context deterministically, and building validation pipelines that verify agent-generated code against historical norms. These are not straightforward engineering tasks—they require systematic investigation into representation design, retrieval-relevance tradeoffs, and trust calibration.

Previous Work

This proposal, developed in collaboration with Ramesh Periyathambi at eBay, builds directly on prior and ongoing work by our team:

Compass (eBay Internal). eBay’s enterprise knowledge graph, Compass, connects Jira tickets to git repositories, commits, PRs, deployments, and microservices. It provides the foundational infra-structure for relating project-management artifacts to code changes—a natural backbone for the agentic KG we propose.

VersiCode. The VersiCode dataset [2] demonstrated the feasibility of encoding version-aware code knowledge in structured form, tracking API evolution across library versions—a capability directly applicable to eBay’s evolving codebase.

PA-AKG (DOE Genesis). Our ongoing DOE-funded project is building provenance-aware agentic knowledge graphs for scientific software [1]. The core architecture—AST-based KG extraction, agentic orchestration with ranking and validation agents, and provenance tracking—directly transfers to eBay’s enterprise context.

FOSE 2026. Our position paper at ICSE-FOSE 2026 articulated the need for structured knowledge accumulation in software engineering [1], providing the intellectual foundation for this proposal.

Methodologies / Approaches

We propose a system with three integrated layers:

1. Code Knowledge Graph Construction. Working with Ramesh and his team at eBay, we will extract structural patterns from eBay’s Java codebase using Abstract Syntax Trees (ASTs) to capture API usage, dependency relationships, and code conventions. Each pattern—a `ViewItem` signal handler, a payment integration, an A/B test variant—becomes a sub-graph of entity nodes (APIs, files, modules) linked by typed edges (`imports`, `implements`, `dependsOn`). We will augment this with historical data from git history, PR descriptions, and code review comments, encoding not just what was built, but why and how it evolved.

2. Pattern Mining & Evolution Tracking. We will mine frequent sub-graph patterns across the codebase, identifying recurring idioms and team-specific conventions. By tracking how these patterns change across commits and releases, the KG will capture pattern evolution—enabling agents to distinguish current best practices from deprecated approaches.

3. Agentic Orchestration. We will deploy three role-specific agents:

- *Ranking agents* score candidate code patterns from the KG by relevance to the current task, filtering noise from large enterprise codebases.
- *Generation agents* produce code conditioned on the ranked patterns, using the KG as structured context rather than relying solely on LLM priors.
- *Validation agents* check generated PRs against KG-encoded patterns, flagging deviations from team conventions or breaking changes in the dependency graph.

This architecture makes code generation **deterministic**—the same task queried against the same KG state produces the same output—and **auditable**—every recommendation traces back to specific historical patterns in the KG.

Impact

If successful, this project will directly reduce the time eBay developers spend on recurring code patterns from weeks to hours, by enabling AI agents that generate production-ready, convention-compliant code on the first attempt. The agentic KG will serve as a persistent organizational

memory—new team members (e.g., offshore teams) can query it to understand project conventions, dependency ownership, and historical design decisions without shadowing senior developers.

The validation pipeline addresses eBay’s code-review bottleneck for AI-generated PRs: instead of manual review of every agentic change, the KG provides automated conformance checking against historical norms, flagging only genuine anomalies for human attention.

Risks. The scale of eBay’s codebase may require sub-sampling; we will mitigate by focusing on well-scoped patterns (e.g., ViewItem signals) as initial case studies. KG extraction reliability is an open problem; we will use staged evaluation with manual validation.

Expected Product / Output / Milestones

Deliverables:

- **PoC Agentic KG** for a scoped subset of eBay’s codebase (ViewItem signals), with pattern mining and agentic orchestration pipeline.
- **Validation framework** that compares agent-generated PRs against KG-encoded patterns and produces conformance reports.
- **Empirical evaluation** measuring reduction in developer time, pattern retrieval accuracy, and PR acceptance rates.
- **Publications** at venues such as ICSE, FSE, and MSR.

Timeline (12 months):

- **Months 1–3:** KG schema design, AST extraction pipeline, initial population from eBay codebase.
- **Months 4–7:** Pattern mining algorithms, ranking agent implementation, pilot study on ViewItem signals.
- **Months 8–10:** Full agentic orchestration (generation + validation), integration with Compass.
- **Months 11–12:** Empirical evaluation, case studies, paper writing.

Metrics: Pattern retrieval precision/recall, agent-generated code acceptance rate, developer time reduction. Tiered funding could support progressive expansion to additional pattern types and teams.

University / Lab / Researcher Qualification

Virginia Tech (PI: Dr. Chris Brown). Dr. Brown leads the Software Engineering and Digital Society lab, researching human-centered AI for software engineering, LLM-based code generation, and developer tooling. He has published at ICSE, FSE, ESEM, and CSCW, and is PI on the DOE Genesis PA-AKG project directly informing this proposal.

Jason Cusati (Graduate Research Assistant, Virginia Tech). Mr. Cusati is a PhD candidate whose research focuses on knowledge graphs for software engineering and AI-assisted code generation. He co-authored the FOSE 2026 position paper on structured knowledge accumulation and is the lead researcher on the PA-AKG project.

References

References

- [1] Jason Cusati and Chris Brown. A case for structured knowledge accumulation in software engineering research. In *Proceedings of the Future of Software Engineering (FOSE) at ICSE*, 2026.
- [2] Tongtong Wu et al. Versicode: Towards version-controllable code generation. *arXiv preprint arXiv:2406.07411*, 2024.